Agentic SOC Investigation Copilot

Full project plan, implementation roadmap, and master coding prompt

Purpose: A substantial portfolio-grade, local-first agentic SOC investigation application.

Budget: ■0 for the core workflow; free/open-source software and optional provider free tiers.

Important design choice: Do not depend on a production SIEM or guaranteed real-time telemetry. Use real input records, deterministic replay, analyst-uploaded logs, and optional local streaming adapters. Do not invent alerts with an LLM.

1. Product definition

Build an **Agentic SOC Investigation Copilot**: an investigation and analyst-interaction layer, not a Splunk replacement. It ingests security events, normalizes and stores them, runs deterministic detection rules, creates evidence-linked alerts, investigates alerts with tool-using agents, checks IP reputation through AbuseIPDB and VirusTotal when configured, lets an analyst chat with agents during an investigation, and produces an evidence-backed report with safe response recommendations.

A successful demo: load a dataset → replay events → deterministic rule creates an alert → open investigation → agents search related events → reputation tools query both providers if configured → analyst asks a question and redirects investigation → verifier checks claims → report and audit record are produced.

Principles

- Local-first: Python, Ollama, SQLite, Streamlit; no hosted LLM required.
- Reproducible: bundled dataset and replay controls always work.
- Real evidence: agents must execute tools and cite stored event/evidence IDs.
- Deterministic detection: Python rules create alerts; the LLM does not invent them.
- Human-in-the-loop: chat, pause/resume/stop/redirect, approve/reject/simulate.
- Graceful degradation: no API keys, SIEM, live stream, or working LLM must not break basic ingestion and detection.
- Safety: no arbitrary shell execution, no automatic blocking, no account/endpoint changes.
- Uncertainty is valid: reputation signals are not proof; “insufficient evidence” is an acceptable verdict.

Non-goals for the initial release

No enterprise SIEM replacement, endpoint agent deployment, guaranteed OS monitoring, production containment, cloud services, paid LLM API, or requirement for real-time external telemetry. Optional Wazuh/Elastic integration comes only after the core works.

2. Data-source plan: do not assume real-time logs exist

Implement multiple input levels, all using the same ingestion service:

1. **Bundled dataset + replay (required):** small, documented JSONL/CSV sample events representing benign, suspicious, and ambiguous authentication, process, and network activity. Replay at selectable speeds; pause, resume, stop, restart. Alerts are generated by rules over these actual records.
2. **File upload (required):** JSONL, NDJSON, CSV, and supported normalized JSON. Preview field mapping; preserve raw records. Quarantine malformed rows with errors.
3. **File tail (optional but planned):** follow appended newline-delimited JSON records from a user-selected local file. Handle partial lines, file rotation/truncation, and stored offsets. Explain that only records written to that file are visible.
4. **Local HTTP event receiver (optional but planned):** `POST /api/v1/events`, bind to `127.0.0.1` by default, validate payloads and size. Never expose publicly by default.
5. **Future SIEM adapter:** define an adapter interface, but do not claim a Wazuh/Elastic adapter exists until implemented and tested.

Do not claim the app automatically reads Windows Event Logs, EDR alerts, or system logs. It only sees data from configured sources. Live telemetry requires permissions, a configured collector, and an available source.

3. Required features

Data and detection

- JSONL/NDJSON/CSV import, explicit field aliases, timestamp normalization to UTC, stable event IDs, raw-record retention, parse-error quarantine.
- SQLite event store, source health, event search/filter, pagination, related-event search and timeline.
- Replay service uses the same path as upload: parse → validate → normalize → deduplicate → persist → detection.
- Versioned, documented, deterministic Python rules:

- repeated failed logins within a configurable time window;

- failed logins followed by successful login for a related user/source key;

- suspicious process/PowerShell event based on explicit testable fields;

- network event matching a local IOC list.

- Optional network-frequency rule only if the dataset supports it.
- Alert deduplication/suppression, rule explanation, exact matched event links, severity, status lifecycle.
- LLM not required for parsing, detections, event search, or basic reports.

Agent investigation

Required roles:

1. **Orchestrator Agent:** manages objective/state, chooses the next step from an allowlist, delegates, considers analyst instructions, stops at limits.
2. **Log Investigation Agent:** searches events, correlates authentication/process/network activity, builds a timeline, reports supporting and contradictory evidence.
3. **Threat Intelligence Agent:** checks local IOC data and optional AbuseIPDB/VirusTotal adapters.
4. **Verification Agent:** validates claims against actual evidence, identifies gaps/conflicts, can return inconclusive.
5. **Response Planner:** proposes proportionate next steps; can be a graph node rather than a separate autonomous agent.

One Ollama model can serve all roles with different prompts. Do not pretend each role requires a separate model. Deterministic code must enforce permissions and policy.

Analyst-agent conversation

- Persistent chat inside each investigation.
- Default chat with Orchestrator; direct specialist selection where useful.
- **Ask mode:** questions and explanations; no workflow change.
- **Investigate mode:** instructions that trigger allowed tools, such as “search this user’s events in the prior 30 minutes.”
- **Control mode:** pause, resume, stop, redirect.
- Log every instruction, tool call, result, and plan change.
- Preserve earlier evidence when redirected; show running/paused/waiting/completed/failed states.
- Limit graph steps, tool calls, runtime and output size; lock/serialize concurrent updates to one investigation.

IP reputation

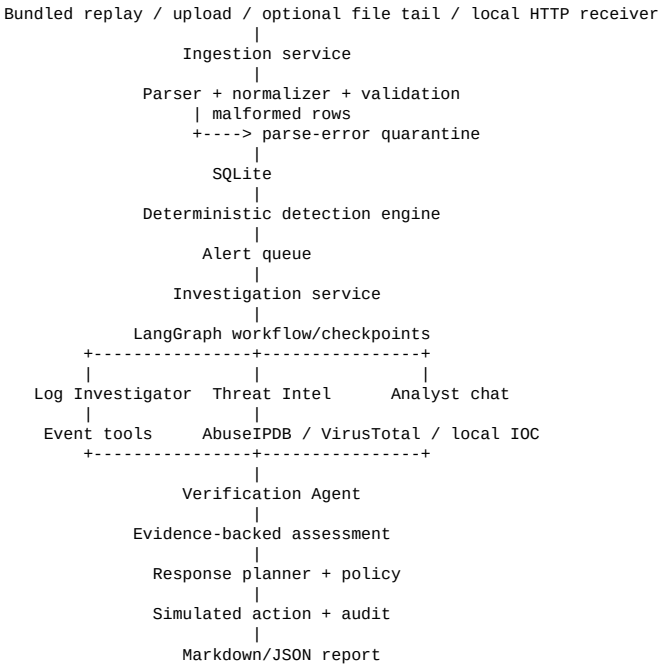
- AbuseIPDB: abuse reports and confidence signal.
- VirusTotal: IP report and vendor-detection/reputation information.
- Local IOC CSV/JSON for offline operation.
- API keys in environment variables or ignored `.env`; `.env.example` contains blank placeholders only.
- External lookups are opt-in, only send the IP indicator needed, validate public IPs and skip private/loopback/link-local/multicast/reserved addresses by default.
- Timeout, bounded retry/backoff, rate-limit handling, sanitized cache in SQLite with provider, indicator, query time, expiry and status.
- Display fresh/cached/not-configured/rate-limited/error status. A provider failure must not fail the investigation.

- Respect API terms, rate limits and free-tier restrictions. Do not query every event automatically or upload whole logs.
- Tests must mock providers and pass without keys or internet.

Evidence, verdict, response and reporting

- Store evidence IDs, source event IDs, timestamps, provenance, summaries and relevant provider lookup references.
- Separate direct observation, inference, conflicting evidence, missing telemetry and recommendation.
- Verification outcomes: `confirmed_malicious`, `likely_malicious`, `suspicious_inconclusive`, `benign_or_expected`, `insufficient_data`.
- Report: executive summary, alert/rule context, timeline, evidence table, provider findings/timestamps, analyst instructions, contradictions/gaps, verdict/confidence explanation, next steps, simulated actions and limitations.
- Export Markdown and JSON.
- Response recommendations have rationale, evidence IDs, risk, reversibility and approval status. All response actions are simulated; no firewall/EDR/IAM integration or destructive action.

4. Architecture and stack



Recommended stack:

- Python 3.11+ (pick and pin a version), type hints.
- Ollama local API; choose model only after checking RAM/VRAM and testing structured output/tool use.
- LangGraph for orchestration/checkpoints; if compatibility is a blocker, use a tested explicit state machine behind a replaceable interface.
- Streamlit UI.
- SQLite via built-in `sqlite3`, migrations/versioned schema, repositories.
- Pydantic validation; `httpx` for provider clients; pytest; Ruff.
- Optional FastAPI only if needed for local HTTP ingestion/API; do not add complexity for its own sake.
- No paid APIs, hosted database, paid SIEM, or cloud storage.

5. Core data model

Use UTC internally and enable SQLite foreign keys. Minimum tables:

- `events`: ID, source ID, source record locator, event time, ingest time, event type, normalized fields JSON, raw record, parse status, schema version.
- `data_sources`: type, enabled/status, last-seen, cursor/offset metadata.

- `parse_errors`: source, row/line, error and safe raw snippet.
- `detection_rules`: ID/version, enabled/configuration.
- `alerts`: rule/version, severity/status, title, dedup key, first/last seen.
- `alert_events`: alert/event links and match reason.
- `investigations`: objective, status, linked alerts, disposition, confidence, report.
- `evidence`: ID, investigation, source type/ID, claim, timestamp, provenance and reliability notes.
- `agent_runs`, `tool_calls`, `chat_messages`: status, role/tool, sanitized arguments/results, timestamps, errors and correlation IDs.
- `reputation_cache`: provider, indicator, checked/expiry time, status, sanitized response.
- `recommendations`: action, rationale, evidence IDs, risk, approval/simulation status.
- `audit_events`: actor, action, target, timestamp and details.
- `app_settings`: non-secret settings only.

Never store API keys in SQLite. Preserve raw event data and use explicit alias mapping; unknown fields stay unknown rather than being guessed.

6. Agent/tool contracts and safety

Allowlisted tools only:

- `search_events(filters)`
- `get_event(event_id)`
- `get_related_events(seed_event_id, time_window, fields)`
- `build_timeline(event_ids)`
- `get_alert_rule_explanation(alert_id)`
- `lookup_local_ioc(indicator)`
- `lookup_abuseipdb(ip)`
- `lookup_virustotal_ip(ip)`
- `add_evidence(...)`
- `request_analyst_input(question)`
- `propose_response(action, rationale, evidence_ids)`

Model output must validate against strict schemas. Deterministic code validates tool names/arguments, authorization, timeouts, step limits and outputs. Never use `eval`, `exec`, model-generated SQL, shell commands or dynamic imports. Treat log messages, command lines, URLs, filenames and provider output as untrusted prompt-injection-capable data. Telemetry cannot override system policy. Never cite an event/evidence ID that the tool did not return. If Ollama is unavailable, data exploration and deterministic detections still work and AI status is clearly marked unavailable.

7. UI pages

1. **Overview:** source status, alert counts, recent alerts, open investigations, agent activity, integration status.
2. **Events explorer:** search, filters, raw/normalized event view, related events.
3. **Alert queue:** severity/status/rule filters, rule explanation, matched events, investigate action.
4. **Investigation workspace:** objective, timeline, evidence, agent activity, chat, verdict, response recommendations, report.
5. **Data sources:** upload, schema preview, replay controls, file-tail status, parse errors.
6. **Detection rules:** enable/disable, version, thresholds, test status and explanation.
7. **Settings/integrations:** Ollama health/model, provider configured status, external lookup toggle, cache status, safe mode.
8. **Audit log:** analyst instructions, tools, approvals, failures, simulated actions.
9. **Report view/export:** Markdown and JSON.

Keep UI usable with no keys, no live source, and Ollama stopped. Show informative status rather than stack traces.

8. Repository layout

```
agentic-soc-copilot/
  README.md  pyproject.toml  .env.example  .gitignore  Makefile
  docs/
    architecture.md  data-sources.md  detection-rules.md
    agent-contracts.md  threat-model.md  api.md  demo-runbook.md
    IMPLEMENTATION_PLAN.md  decisions/
  app/
    main.py  config.py
    ui/ (dashboard, events, alerts, investigation, sources, settings, audit)
    api/ (routes, schemas)
    domain/ (models, enums, errors)
    ingestion/ (parsers, normalizer, upload, replay, file_tail, http_receiver)
    storage/ (db, migrations, repositories)
    detection/ (engine, registry, rules)
    investigations/ (service, graph, state, prompts)
    agents/ (orchestrator, log_investigator, threat_intel, verifier, response_planner)
    tools/ (event_search, reputation/abuseipdb, reputation/virustotal, local_ioc, schemas)
    policy/ evidence/ reporting/ audit/
  data/demo/ (manifest.json, auth_events.jsonl, endpoint_events.jsonl,
              network_events.jsonl, local_iocs.csv)
  scripts/ (check_ollama.py, validate_dataset.py, run_demo.py)
  tests/ (unit, integration, security, fixtures, end_to_end)
```

Do not create meaningless empty modules. Build cohesive parts in stages.

9. Implementation roadmap

Phase 0 — Environment: inspect OS/Python/RAM/disk/Ollama; choose a model based on hardware; create project metadata, `.gitignore`, `.env.example`, architecture notes. Do not download a large model without confirming expected size.

Phase 1 — Data foundation: Pydantic event schema, SQLite schema/migrations/repositories, JSONL/NDJSON/CSV parsers, normalizer, ingestion service, quarantine, search, demo dataset/manifest, event explorer.

Phase 2 — Replay and detection: replay controls through shared ingestion; authentication rules; process and IOC rules; dedup/suppression; alert lifecycle/queue/explanations.

Phase 3 — Reputation: provider interface, AbuseIPDB, VirusTotal, local IOC, caching, rate limits, settings/status page, mocked tests.

Phase 4 — Investigation/evidence: investigation lifecycle, search/timeline tools, evidence provenance, deterministic report generation before adding agents.

Phase 5 — Agent graph: orchestrator, log investigator, threat intel, verifier; structured prompts/outputs, allowlisted tools, checkpoints, timeouts and step limits.

Phase 6 — Analyst chat: persistent chat, Ask/Investigate/Control modes, specialist selection, pause/resume/stop/redirect, visible activity, per-investigation concurrency control.

Phase 7 — Policy/audit/reporting: response recommendations, simulate/approve/reject, deterministic policy gate, audit trail, Markdown/JSON export.

Phase 8 — Hardening: integration and end-to-end tests, file/payload limits, UI escaping, secrets redaction, offline/error states, clean install testing, demo runbook.

Phase 9 — Optional integrations: syslog, Wazuh/Elastic adapter, richer Windows event import, PDF report. Only after core acceptance tests pass.

At the end of every phase: run tests and lint; fix failures; update docs and checklist; report exactly what works and what remains. Do not claim tests passed unless they actually ran.

10. Acceptance checklist

- ■ Clean install documented and repeatable.
- ■ JSONL/NDJSON/CSV import, field mapping, malformed-row quarantine.
- ■ Replay/upload/file-tail/HTTP use shared ingestion path.
- ■ Replay pause/resume/restart avoids duplicate ingestion.

■ Same dataset produces reproducible deterministic alerts with event links.

■ Detection works without Ollama.

■ Both provider adapters validate IPs, handle errors/rate limits, cache results and hide keys.

■ Tests pass with mocked APIs and no internet/API keys.

■ Investigation searches related events and records evidence provenance.

■ Agents execute real allowlisted tools; no invented tool results.

■ Analyst can chat, redirect, pause/resume/stop, and preserve previous evidence.

■ Verifier can return inconclusive and shows contradictions/missing telemetry.

■ Policy prevents real high-impact actions; only simulations are implemented.

■ Audit log and Markdown/JSON reports work.

■ Prompt injection in telemetry does not override policy.

■ README explains data privacy, API keys, provider limits, troubleshooting and demo steps.

11. Repeatable demo scenario

Use safe sample indicators and mocked provider responses in automated tests; do not depend on a particular real IP having a malicious reputation.

1. Replay benign logins, repeated failures and a later success.
2. Deterministic rule creates an alert linked to exact events.
3. Start investigation; Log Investigator searches surrounding events.
4. Threat Intel checks local IOC and both providers if configured.
5. Analyst asks: “Could this be legitimate? Find evidence for and against compromise.”
6. Analyst redirects: “Search the same user's events for 30 minutes before and after.”
7. Verifier checks evidence and may return suspicious/inconclusive.
8. Response Planner proposes safe next steps; simulate or reject.
9. Export report and show audit trail.

12. Configuration and operating assumptions

`.env.example` should contain placeholders, never real secrets:

```
OLLAMA_BASE_URL=http://127.0.0.1:11434
OLLAMA_MODEL=your-locally-installed-model
ABUSEIPDB_API_KEY=
VIRUSTOTAL_API_KEY=
DATABASE_PATH=./data/agentic_soc.db
APP_HOST=127.0.0.1
APP_PORT=8501
ENABLE_EXTERNAL_REPUTATION=false
```

The application must start without provider keys. External lookups remain disabled until enabled. The app must explain that external reputation requests send the queried IP to the provider. Respect provider terms, especially free/public API restrictions.

13. MASTER CODING PROMPT — copy everything inside this section

You are the lead Python engineer and security architect implementing **Agentic SOC Investigation Copilot** in the current repository. Build a real, testable, local-first application—not a toy chatbot, not an LLM-generated alert demo, and not a SIEM replacement.

Hard constraints

- Core workflow costs ■0: Python, SQLite, Streamlit, Ollama and open-source dependencies. No paid LLM API, hosted database, paid SIEM or cloud storage.
- Use Ollama for local inference. Do not send prompts/logs to hosted LLMs.

- AbuseIPDB and VirusTotal are optional integrations. They must work when keys are configured and fail gracefully without keys. Respect provider terms and limits.
- Do not assume live telemetry. Bundled dataset + deterministic replay is the reliable default. Also support file upload; add optional JSONL tail and loopback-only HTTP ingestion. Do not claim automatic OS log access.
- Alerts must be created by deterministic Python rules over actual input records, never by asking an LLM to invent alerts.
- Use SQLite and a versioned schema/migrations. Use LangGraph if compatible; otherwise use a tested state machine behind a replaceable interface.
- Support interactive chat with Orchestrator and specialist agents, additional investigation instructions, pause/resume/stop/redirect and persistent state.
- No arbitrary shell/Python/SQL execution from model output. No real blocking, account disabling or EDR actions; response actions are recommendation/simulation only.
- Use only synthetic/publicly licensed/demo data. Never commit workplace/customer logs or secrets.
- Build a modular monolith; no unnecessary microservices. Do not claim a feature works until implemented and tested.

Agents

1. Orchestrator: maintains goal/state, chooses allowed next step, delegates, handles analyst instructions and stopping criteria.
2. Log Investigator: searches events, correlates auth/process/network activity, builds timeline, reports supporting and contradicting evidence.
3. Threat Intelligence: local IOC + AbuseIPDB + VirusTotal, with provenance, timestamps, cache state and provider errors.
4. Verifier: validates every claim against actual event/evidence/tool results, identifies conflicts/gaps and can return inconclusive.
5. Response Planner: recommends proportionate next steps, with evidence/risk; all actions simulated and policy-gated.

One Ollama model may serve multiple roles using distinct prompts. Do not claim separate models are needed.

Ingestion

Implement one shared pipeline for upload/replay/file-tail/HTTP: parse → validate → normalize → deduplicate → persist → evaluate rules → create/link alerts → update source health. Preserve raw data and provenance; quarantine malformed rows. Support JSONL/NDJSON/CSV with explicit field alias mapping and preview. Store event time and ingestion time in UTC. Replay uses the same ingestion path, with speed, pause, resume, stop and duplicate-safe restart. File tail handles partial lines/rotation/truncation. HTTP receiver binds to 127.0.0.1 by default, validates payload size/schema, and is never publicly exposed by default. Design a future SIEM adapter interface but do not pretend one is implemented.

Detection

Implement and test versioned deterministic rules for repeated failed logins; failures followed by success; suspicious process/PowerShell telemetry based on explicit fields; and network events matching local IOC data. Optional rules only if data supports them. Alerts must include rule/version, severity, explanation, dedup key and exact matching event IDs. Add tests for missing fields, benign lookalikes, time boundaries, duplicates and replay. Detection works without Ollama.

Reputation

Create `AbuseIPDBProvider` and `VirusTotalProvider` behind a common interface. Read keys only from environment/ignored `.env`. Validate IPs and skip private, loopback, link-local, multicast, reserved and invalid addresses by default. Use timeouts, bounded retries/backoff, rate-limit handling, sanitized SQLite cache and expiry. External lookup is opt-in and sends only the queried indicator, not entire logs. Do not query every event automatically. Show not-configured/cached/fresh/rate-limited/timeout/provider-error states. Mock provider calls in tests.

Tools and guardrails

Allow only schema-validated tools: `search_events`, `get_event`, `get_related_events`, `build_timeline`, `get_alert_rule_explanation`, `lookup_local_ioc`, `lookup_abuseipdb`, `lookup_virustotal_ip`, `add_evidence`, `request_analyst_input`, `propose_response`. Deterministic code validates tool name, arguments and authorization, executes it, records the result and returns sanitized output. Never eval/exec, run shell commands, execute model-generated SQL or dynamically import based on model output. Enforce maximum graph steps/tool calls, timeouts, output limits and per-investigation concurrency control. Treat telemetry/provider responses as untrusted prompt-injection-capable data. Event contents never override system policy. If Ollama fails, deterministic event/detection functions still work.

Interactive analyst UI

Use Streamlit pages: dashboard; events explorer; alert queue/detail; investigation workspace (timeline, evidence, agent activity, chat, verdict and recommendations); data sources/upload/replay; detection rules; settings/integration health; audit log; report export. Chat has Ask (read-only), Investigate (allowed tools), Control (pause/resume/stop/redirect) modes and optional specialist selection. Persist all

messages/instructions; a redirect updates workflow state and preserves prior evidence. Show truthful running/paused/waiting/completed/failed status.

Evidence and reports

Every factual claim links to valid stored evidence IDs. Separate observed facts, inferences, contradictions, missing telemetry and recommendations. Allowed dispositions: `confirmed_malicious`, `likely_malicious`, `suspicious_inconclusive`, `benign_or_expected`, `insufficient_data`. Export Markdown and JSON reports with summary, alert/rule context, timeline, evidence, provider results/timestamps, analyst instructions, gaps, verdict reasoning, recommendations, simulated actions and limitations. Do not invent numerical confidence calibration.

Security and engineering

Python 3.11+; type hints; Pydantic; SQLite foreign keys; pytest; Ruff; structured logs/correlation IDs; `.env.example`; `.gitignore`; secret redaction; file/payload limits; UI escaping; prompt files under version control. Mock external API calls. Include `docs/architecture.md`, `data-sources.md`, `detection-rules.md`, `agent-contracts.md`, `threat-model.md`, `demo-runbook.md`, and `IMPLEMENTATION_PLAN.md`. Keep logic out of UI callbacks. Do not create meaningless empty modules.

Work iteratively—do not generate the entire project in one giant code dump.

First inspect repository and environment: files, Python version, RAM/disk if accessible, Ollama availability and existing code. Do not overwrite existing work without inspection. Then create `docs/IMPLEMENTATION_PLAN.md`. Implement these phases in order:

0. Environment/config/docs.
1. Domain models, SQLite schema/migrations, repositories, parsers, normalization, ingestion, quarantine, search, demo data and event explorer.
2. Replay engine and deterministic rules/alert queue.
3. AbuseIPDB/VirusTotal/local IOC, cache/settings/mocked tests.
4. Investigation/evidence/timeline/report services.
5. Agent graph, role prompts, structured tool contracts and limits.
6. Persistent analyst chat, redirect/pause/resume/stop and state locking.
7. Response policy, simulations, audit and report exports.
8. Optional file tail and loopback HTTP receiver.
9. Hardening, end-to-end tests, README and demo runbook.

At the start of each phase, state scope/files/risks. At the end, run tests and lint, fix failures, update docs and checklist, and report what works, actual commands/tests and limitations. Never claim tests passed unless they ran. If you cannot run commands, say so and provide exact commands. Ask the user only for real decisions (such as model/hardware choice or destructive changes); otherwise choose sensible free defaults and document them. Begin by inspecting the environment and implementing Phase 0/Phase 1 only. Do not proceed to later phases until those tests are reviewed.

Completion requires: clean setup; replay; deterministic event-linked alerts; optional provider integrations; real allowlisted tool use; interactive agent conversation and redirection; evidence-backed verifier including inconclusive outcomes; simulated responses; audit and exports; unit/integration/security/end-to-end tests; clear README and privacy limitations; no paid dependency for core demo. Begin with repository/environment inspection and a concise assessment.

14. How to use this plan

1. Create/open a dedicated repository.
2. Install Python and Ollama; choose a model after checking available RAM/VRAM.
3. Paste the master prompt into your repository-aware coding agent.
4. Let it inspect the repository and implement only Phase 0/1.
5. Review changes and run tests. Continue phase-by-phase; do not accept untested claims.
6. Configure AbuseIPDB/VirusTotal only when you have keys; core app must still run without them.
7. Get permission before importing workplace logs.
8. Rehearse the demo from a clean database.

Recommended order: ingestion → SQLite → replay → deterministic detection → alert queue → reputation integrations → evidence/investigations → agents → interactive chat → policy/audit/reporting → optional live adapters.